QPS TRIAGE · ABACUS · CODEX — EXECUTION SEED 2026-10-12 · FEDERATED DMAIC

Global Code Purple

A reusable, recursively self-testing execution protocol across repositories, agents, Docker, CI, PR admission and evidence. It builds on QPS TRIAGE, ABACUS, CODEX, KEB and DOW and does not introduce a second system of authority.

NOT LIFTED · 2 of 5 lift criteria | read back 2026-10-18 13:17 UTC | ABACUS #1858 · merged | ABACUS #1859 · merged | ABACUS: 12 open PRs | CODEX: 0 open PRs | 97 merges since unblock

THE CENTRAL RULE · SELECT A STEP TO TRACE IT ON THE PULSE GRAPH

01
Reuse proven evidence
Define → Measure

02
Allocate, preconfigure and pretest
Measure

03
Identify the first material red
Analyse

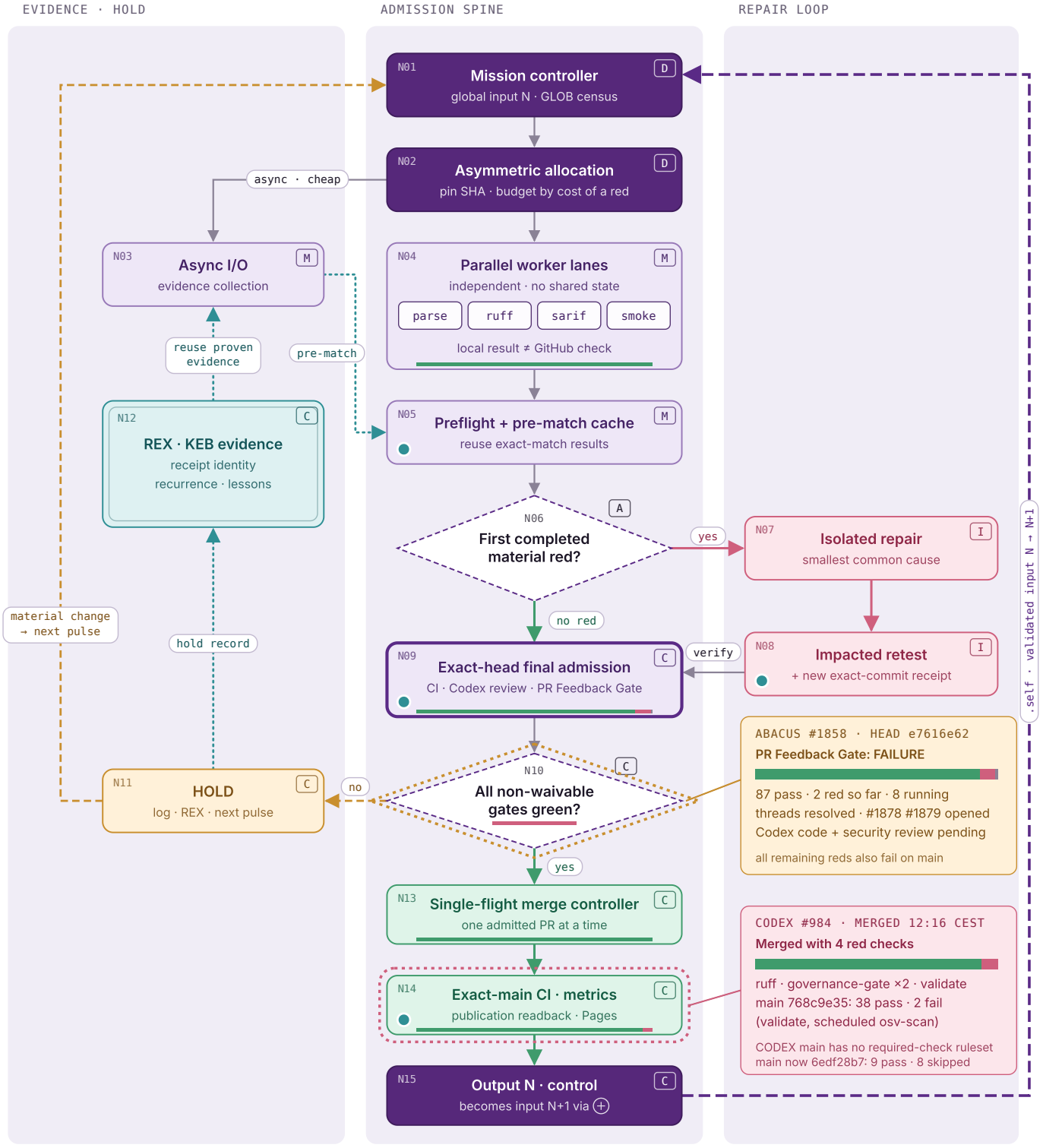
04
Repair the smallest common cause
Improve

05
Independently verify
Improve

06
Publish exact-head evidence
Control

07
Advance only admissible successors
Control

1 · Execution pulse



Legend

NODE ROLES

- Authority · resolves or owns state
- Process · checks and verification
- Gate · binary decision
- Repair · isolated change
- Admitted path · merge and proof
- Hold · blocked, logged, retried
- Evidence store · REX and KEB

EDGE TYPES

- Sequence
- Pass
- Material red
- Hold
- Evidence
- .self recursion

MARKERS

- `C` DMAIC phase · hover for card
- `N09` Node ID · hover for card
- Run bar: pass · fail · pending
- Emits an exact-commit receipt
- Where a tracked PR is now

`D Define` `M Measure` `A Analyse` `I Improve` `C Control`

2 · Census and telemetry

Iteration census from the `.self` harness: preconfig state per node, run attempts and outcomes, errors, logs, REX lessons, Python runtime and version control. Pass rate counts decisive runs only; pending and neutral runs are shown separately.

ATTEMPTS	PASSED	FAILED	PENDING	PASS RATE	ERRORS
221	207	13	1	94%	5
iteration 0	decisive green	decisive red	1 neutral	of decisive runs	0 REX lessons

Coverage · breadth × depth

local run with the CI command on #1858 head 203d63af · 1,086 tests

OVERALL	LINES	BREADTH	DEPTH	UNTESTED FILES	GATE
36.9%	38.2%	64.5%	59.2%	114	75%
line + branch	10,686 / 27,998	155 of 269 files touched	inside touched files	57% of the gap	fixed; red on every PR

WAVE	SCOPE	LINE COVERAGE AFTER	TARGET DATE (PROPOSED)
W0	Floor = measured; new and changed lines ≥ 80% (diff coverage)	38.2%	16 Oct
W1	Written disposition for 34 one-shot or legacy files (keep+test, archive, omit)	46.5%	20 Oct
W2	DMAIC_V3 phase modules to 80%	52.1%	27 Oct
W3	53 untested DMAIC_V3 modules to 70%	65.5%	6 Nov
W4	34 partly covered DMAIC_V3 modules to 75%	72.1%	13 Nov
W5	tools, scripts, models, src, renderers, integration to 80%	80.2%	20 Nov

Version control

REPO	GBOGEB/CODEX · GBOGEB/ABACUS
BRANCH	#984 merged · #1858 fix/gate-owner-attestation-fallback
HEAD	ABACUS #1858 203d63aff64b9d6b4c69dec0a1d3f5cf3f1f8bf1
BASE	CODEX main 6edf28b7 · ABACUS main 53688f26 (#1838, 2026-10-06)
DIRTY TREE	not reported
GENERATED	2026-10-12T15:16:00Z (GitHub API readback)

Runtime and debug

PYTHON	3.10 · 3.11 · 3.12 (ABACUS CI matrix, all green on #1858)
IMPLEMENTATION	CPython (GitHub-hosted runners)
PLATFORM	ubuntu-24.04
CONTAINER	not reported
DEBUG DEFAULT	CODEX main: LLDB, lldb-dap + MCP, debugpy + js-debug lanes green
PER-NODE	not reported

Node census

ID	NODE	STAGE	TEST	DEBUG	ATT	✓	✗	PEND	PASS	RUNS	LAST ERROR
N01	Mission controller	pass	none	–	—	—	—	—	—		—
N02	Asymmetric allocation	–	–	–	—	—	—	—	—		—
N03	Async I/O	–	–	–	—	—	—	—	—		—
N04	Parallel worker lanes	pass	pass	on	15	15	0	0	100%		—
N05	Preflight + pre-match cache	–	–	–	—	—	—	—	—		—
N06	First completed material red?	–	–	–	—	—	—	—	—		—
N07	Isolated repair	–	–	–	—	—	—	—	—		—
N08	Impacted retest	–	–	–	—	—	—	—	—		—
N09	Exact-head final admission	pass	pass	–	155	144	10	1	94%		PUSH-203d63af New head (workflow_policy.json +1): Code Coverage, Code Quality, Full Red Proof, Security Scanning, policy-and-inventory red
N10	All non-waivable gates green?	–	–	–	1	0	1	0	0%		NO-GATE CODEX main has no required-check ruleset; #984 merged without a green admission
N11	HOLD	–	–	–	—	—	—	—	—		—
N12	REX · KEB evidence	–	–	–	—	—	—	—	—		—
N13	Single-flight merge controller	–	–	–	1	1	0	0	100%		—
N14	Exact-main CI · metrics	–	–	–	49	47	2	0	96%		MAIN-768c9e35 validate and scheduled osv-scan failed after the #984 merge; not rerun on later heads
N15	Output N · control	–	–	–	—	—	—	—	—		—

Logs

10-12 17:01	INFO	N02	Pinned heads: ABACUS #1858 10fdaaa5; CODEX main 6edf28b7
10-12 17:01	WARN	N11	Nothing merged to ABACUS main since #1838 (2026-10-06)
10-12 16:56	ERROR	N09	ABACUS #1858 PR Feedback Gate: state=failure, ci_seen=15
10-12 16:56	WARN	N10	ABACUS gate runs main's code; #1858's own fallback cannot admit #1858 (no self-approval)
10-12 13:11	INFO	N09	ABACUS #1858: 3 duplicate gate evaluations cancelled, 1 succeeded
10-12 13:10	WARN	N11	#1858 threads L599 and outage correlation: owner residual-risk replies, unresolved
10-12 12:16	INFO	N01	Code Purple execution seed merged to CODEX main via #984 (12 commits)
10-12 12:16	WARN	N10	CODEX #984 merged with red checks: admission step bypassed
10-12 12:09	ERROR	N09	CODEX #984: 4 failing checks on head 90e3396a

Errors

TIME	NODE	CODE	MESSAGE	REX
10-12 12:09	N09	CODEX-984	ruff, governance-gate ×2 and validate failed on merge head 90e3396a	–
10-12 16:56	N09	GATE-1858	PR Feedback Gate failure: 2 unresolved threads; Codex code and security review pending	–
10-12 17:14	N09	PUSH-203d63af	New head (workflow_policy.json +1): Code Coverage, Code Quality, Full Red Proof, Security Scanning, policy-and-inventory red	–
10-12 12:16	N10	NO-GATE	CODEX main has no required-check ruleset; #984 merged without a green admission	–
10-12 12:20	N14	MAIN-768c9e35	validate and scheduled osv-scan failed after the #984 merge; not rerun on later heads	–

REX lessons

No REX lessons captured yet. Capture starts once controls execute.

▼ **Census JSON contract · code-purple.census/v1**

The `.self` harness writes one file per iteration in this shape. Load it with "Load census JSON" to replace the recorded data on every view.

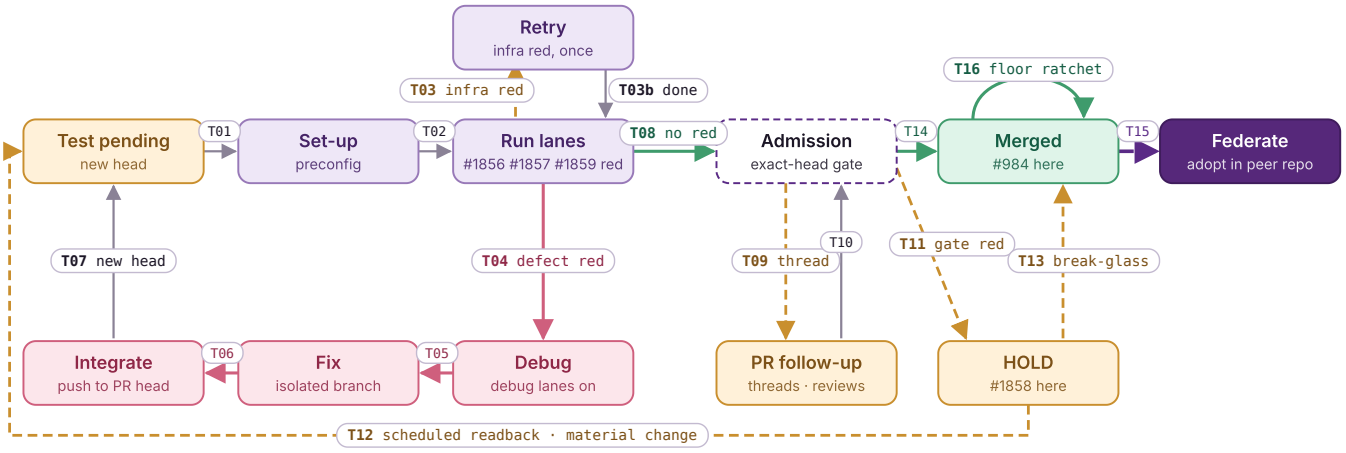
```
{
  "schema": "code-purple.census/v1",
  "iteration": 1,
  "generated_at": "2026-10-16T09:00:00+02:00",
  "vcs": {
    "repo": "GBOGEB/CODEX",
    "branch": "<branch>",
    "head": "<40-char sha>",
    "base": "main@<sha>",
    "dirty": false
  },
  "runtime": {
    "python": "3.11.9",
    "implementation": "CPython",
    "platform": "linux-x86_64",
    "container": "<image or devcontainer>",
    "debug": "off | on | trace"
  },
  "rex": [
    {
      "id": "REX-…",
      "node": "N04",
      "title": "<reusable lesson>",
      "recurrence": 1
    }
  ],
  "nodes": {
    "N04": {
      "preconfig": {
        "stage": "pass | fail | pending | none",
        "test": "pass | fail | pending | none",
        "debug": "off | on | trace"
      },
      "runtime": {
        "python": "3.12.4"
      },
      "runs": [
        {
          "name": "ruff",
          "attempts": 3,
          "pass": 2,
          "fail": 1,
          "neutral": 0,
          "pending": 0,
          "ms": 1800
        }
      ],
      "errors": [
        {
          "ts": "<iso>",
          "code": "RUF100",
          "msg": "<message>",
          "rex": "REX-…"
        }
      ],
      "logs": [
        {
          "ts": "<iso>",
          "level": "ERROR | WARN | INFO | DEBUG",
          "msg": "<message>"
        }
      ]
    }
  }
}
```

3 · Process reference

The pulse as a written procedure. Stages run in order; each lists entry criteria, actions and exit outputs, with the nodes it touches. The node and edge tables are generated from the same model as the graph.

Lifecycle triggers

What moves a work item (a PR head or a red finding) from one state to the next. Each transition has one triggering event, a guard that must hold, and an action that writes evidence. Today only T01's CI start is wired, because GitHub runs checks on pull-request events; the rest are proposed and need the `.self` harness and a readback workflow.



ID	FROM → TO	TRIGGER EVENT	GUARD	ACTION AND EVIDENCE	NODES	WIRED TODAY
T01	Test pending → Set-up	Pull request opened or synchronised, or a push	Head SHA pinned	Start CI; mark the gate pending	N01 N02	Partly wired GitHub starts CI on these events. ABACUS's gate re-evaluates on workflow_run and review events, not on synchronize (#1858 thread, line 599).
T02	Set-up → Run lanes	Preconfig receipt written	Python runtime, dependencies and devcontainer resolved	Start the parallel lanes	N03 N04	Proposed Needs the .self harness.
T03	Run lanes → Retry	A lane goes red with class runner or infrastructure (5xx, runner lost, cancelled)	No earlier retry on this SHA	Re-run the failed jobs once; log a REX entry	N05 N06	Manual Re-run all jobs button on GitHub.
T03b	Retry → Run lanes	Retry finished	—	Results re-enter triage	N06	Manual Same re-run.
T04	Run lanes → Debug	Red classified as a source defect that doesn't reproduce locally	It is the first completed material red	Re-run that lane with debug on (debugpy, LLDB) and keep the trace	N06	Partly wired CODEX main already runs LLDB, lldb-dap and debugpy lanes (green); nothing triggers them from a red yet.
T05	Debug → Fix	Root cause recorded	Fix scope is the smallest common cause	Branch from the exact head	N07	Manual —
T06	Fix → Integrate	Impacted retest green, with an exact-commit receipt	No governance change approves itself	Push to the PR head	N08	Manual —
T07	Integrate → Test pending	New head on the PR	—	Old receipts retired; the cycle restarts at T01	N01	Wired Push event.
T08	Run lanes → Admission	All lanes terminal	No material red	Exact-head CI plus Codex code and security review	N09	Partly wired CI is automatic. The Codex review request is manual, and the allowance is exhausted.
T09	Admission → PR follow-up	Review thread, Codex comment or gate failure	—	Answer, fix, or record an owner residual-risk decision	N09 N11	Partly wired Detection is wired: pr-feedback-gate.yml runs on pull_request_review and review-comment events. The response is manual.
T10	PR follow-up → Admission	Thread resolved or review submitted	Zero unresolved threads	The gate re-evaluates	N10	Wired Review events trigger the gate.
T11	Admission → HOLD	A non-waivable gate is red or pending (e.g. Codex outage)	—	Write the HOLD record and a REX entry	N10 N11 N12	Partly wired The gate holds. The REX record is manual.
T12	HOLD → Test pending	Scheduled readback, or a material change	Something changed since the last pulse	Start a new pulse	N11 N01	Proposed Needs a scheduled task (burndown item 7).
T13	HOLD → Merged	Owner decision with a written residual risk	Owner only. One-time ruleset bypass, restored straight after the merge	Squash-merge, restore the ruleset, write REX	N10 N13	Manual Owner edits ruleset 24494381. This is the proposed path for #1858.
T14	Admission → Merged	All non-waivable gates green on the same SHA	Single-flight: no other merge in progress	Squash-merge	N13	Partly wired ABACUS: ruleset enforces it. CODEX: no ruleset; #984 merged with 4 reds.
T15	Merged → Federate	Exact-main CI green and read back	Consumer listed in MESH_DOCK	Open an adoption PR in the peer repo; write the census row	N14 N15	Proposed Not built. For #984, CODEX main has 2 post-merge reds to clear first.
T16	Merged → Merged	Exact-main CI measures coverage at or above floor + 1 point	Exact-main CI green; measurement on the same SHA	Raise the coverage floor to the measured value; write the census row (total, breadth, depth, per area)	N14 N12	Proposed Not built. Today the gate is a fixed 75% and fails on every PR.

Process stages

1 Mobilise

Define

`N01 · Mission controller` `N02 · Asymmetric allocation`

ENTRY
- A trigger: new head, schedule or material change

ACTIONS
- Census authorities and capabilities
- Pin the exact SHA
- Budget lanes by the cost of a red

EXIT
- Pinned SHA pair
- Lane budget

2 Collect and run

Measure

`N03 · Async I/O` `N04 · Parallel worker lanes`

ENTRY
- SHA pinned, lanes budgeted

ACTIONS
- Collect prior evidence asynchronously from REX, KEB and GitHub
- Run parse, Ruff, SARIF and smoke lanes in parallel

EXIT
- Evidence bundle
- Per-lane results

3 Pre-match

Measure

`N05 · Preflight + pre-match cache`

ENTRY
- Lane results and evidence bundle

ACTIONS
- Reuse results proven for the identical commit and inputs
- Run only the remainder

EXIT
- Preflight receipt (local, not CI proof)

4 Triage

Analyse

`N06 · First completed material red?`

ENTRY
- Preflight receipt

ACTIONS
- Rank findings with DOW
- Assign one of four failure classes
- Select the first completed material red

EXIT
- One red to repair, or none

5 Repair loop

Improve

`N07 · Isolated repair` `N08 · Impacted retest`

ENTRY
- One classified red

ACTIONS
- Fix the smallest common cause on an isolated branch
- Retest the impacted set
- Write a new exact-commit receipt

EXIT
- Repaired head submitted for verification

6 Admission

Control

`N09 · Exact-head final admission` `N10 · All non-waivable gates green?`

ENTRY
- Exact head with no known material red

ACTIONS
- Run CI, Codex review and PR Feedback Gate on the exact SHA
- Evaluate non-waivable gates only

EXIT
- Admit, or hold

7 Hold and learn

Control

`N11 · HOLD` `N12 · REX · KEB evidence`

ENTRY
- A non-waivable gate red or pending

ACTIONS
- Record the reason in REX
- Update recurrence and lessons
- Plan the next pulse

EXIT
- Hold record
- Next pulse on material change

8 Merge and prove

Control

`N13 · Single-flight merge controller` `N14 · Exact-main CI · metrics` `N15 · Output N · control`

ENTRY
- Admission green

ACTIONS
- Merge one PR at a time
- Run exact-main CI, record metrics, read back the publication
- Write the census row

EXIT
- Output N becomes input N+1

Node reference

15 of 15 nodes

ID	NODE	TYPE	PHASE	LANE	IN	OUT	ARTEFACT	EVIDENCE	STATUS
N01	**Mission controller** Takes global input N, discovers authorities and capabilities, and resolves producer, consumer, receipts and scope for this pulse.	Authority	Define	Admission spine	N11 N15	N02	CODEX GLOB.yaml, .mesh/MESH_DOCK.yaml	Census snapshot	Seeded / commi
N02	**Asymmetric allocation** Pins the exact head under test and spends compute where a red is most expensive. Cheap I/O runs async; costly checks get dedicated lanes.	Authority	Define	Admission spine	N01	N04 N03	Seed YAML execution contract	Pinned SHA pair, lane budget	Seeded / commi
N03	**Async I/O** Collects existing evidence without blocking the workers, including prior receipts that may already prove a result.	Process	Measure	Evidence · hold	N02 N12	N05	REX and KEB lookups, GitHub check readback	Evidence bundle	Not implemente
N04	**Parallel worker lanes** Runs parse, Ruff, SARIF security scans and smoke in parallel with no shared state.	Process	Measure	Admission spine	N02	N05	ABACUS scripts/testpilot_preflight.py, pre-commit, Ruff, Bandit, Semgrep	Per-lane JSON and SARIF	Partial
N05	**Preflight + pre-match cache** Merges worker output with collected evidence. A result already proven for the identical commit and inputs is reused instead of rerun.	Process	Measure	Admission spine	N04 N03	N06	Receipts keyed by exact commit and input hash	Preflight receipt	Not implemente
N06	**First completed material red?** Ranks findings and selects the first completed red that matters, classified as source defect, inherited debt, review capacity or runner failure.	Gate	Analyse	Admission spine	N05	N07 N09	ABACUS DOW	Ranked disposition	Not implemente
N07	**Isolated repair** Fixes the smallest common cause only. Security-sensitive governance changes cannot approve themselves.	Repair	Improve	Repair loop	N06	N08	Smallest-scope branch	Commit	Not implemente
N08	**Impacted retest** Reruns only what the repair touches and writes a new exact-commit receipt.	Repair	Improve	Repair loop	N07	N09	Impacted test set	Exact-commit receipt	Not implemente
N09	**Exact-head final admission** Independent verification on the exact head SHA, including required reviews.	Process	Control	Admission spine	N08 N06	N10	GitHub Actions, PR Feedback Gate, Codex review and security review	Check records, review states	In progress
N10	**All non-waivable gates green?** Admits only when every non-waivable check and review is green on the same SHA. Waivable checks are recorded, not counted.	Gate	Control	Admission spine	N09	N11 N13	Required checks and reviews at exact head	Admission decision	On hold
N11	**HOLD** Logs the blocking reason to REX and plans the next pulse. A material change re-enters at the mission controller.	Hold	Control	Evidence · hold	N10	N12 N01	REX entry	Hold record	On hold
N12	**REX · KEB evidence** Holds evidence identity, recurrence counts and reusable lessons. Async I/O reads from it at the start of each pulse.	Evidence store	Control	Evidence · hold	N11	N03	CODEX KEB and REX	Lessons, recurrence	Partial
N13	**Single-flight merge controller** Merges one admitted PR at a time so no head goes stale behind another merge.	Admitted path	Control	Admission spine	N10	N14	Merge queue discipline	Merge commit	Not reached
N14	**Exact-main CI · metrics** Proves main is green after the merge, records metrics and reads the published result back.	Admitted path	Control	Admission spine	N13	N15	GitHub Actions on main, Pages	Post-merge receipt	Not reached
N15	**Output N · control** Becomes the input of iteration N+1 through governed reconciliation (⊕).	Authority	Control	Admission spine	N14	N01	Iteration census	Census row	Not reached

Edge reference

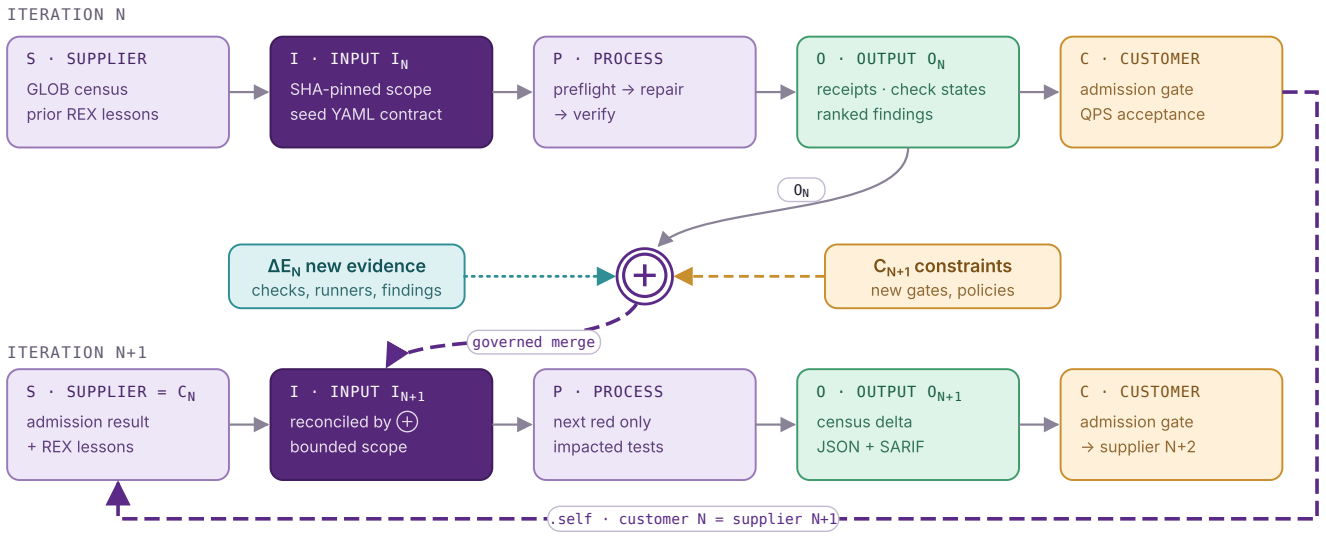
ID	FROM	TO	TYPE	LABEL	MEANING
E01	N01 Mission controller	N02 Asymmetric allocation	Sequence	—	Every pulse starts at the controller.
E02	N02 Asymmetric allocation	N04 Parallel worker lanes	Sequence	—	Costly checks get dedicated lanes.
E03	N02 Asymmetric allocation	N03 Async I/O	Sequence	`async · cheap`	Cheap I/O runs without blocking the workers.
E04	N04 Parallel worker lanes	N05 Preflight + pre-match cache	Sequence	—	Per-lane JSON and SARIF results.
E05	N03 Async I/O	N05 Preflight + pre-match cache	Evidence	`pre-match`	Prior receipts offered for exact-match reuse.
E06	N05 Preflight + pre-match cache	N06 First completed material red?	Sequence	—	Merged preflight result.
E07	N06 First completed material red?	N07 Isolated repair	Material red	`yes`	A completed material red exists.
E08	N07 Isolated repair	N08 Impacted retest	Material red	—	Repair committed on an isolated branch.
E09	N08 Impacted retest	N09 Exact-head final admission	Sequence	`verify`	Retest receipt goes to exact-head verification.
E10	N06 First completed material red?	N09 Exact-head final admission	Pass	`no red`	No material red among completed checks.
E11	N09 Exact-head final admission	N10 All non-waivable gates green?	Sequence	—	Check and review states on the exact head.
E12	N10 All non-waivable gates green?	N11 HOLD	Hold	`no`	A non-waivable gate is red or pending.
E13	N10 All non-waivable gates green?	N13 Single-flight merge controller	Pass	`yes`	Every non-waivable gate is green on the same SHA.
E14	N13 Single-flight merge controller	N14 Exact-main CI · metrics	Pass	—	One PR merged at a time.
E15	N14 Exact-main CI · metrics	N15 Output N · control	Pass	—	Main green; metrics and publication read back.
E16	N11 HOLD	N12 REX · KEB evidence	Evidence	`hold record`	Blocking reason recorded.
E17	N12 REX · KEB evidence	N03 Async I/O	Evidence	`reuse proven evidence`	Lessons and receipts reused in the next pulse.
E18	N11 HOLD	N01 Mission controller	Hold	`material change → next pulse`	A material change triggers the next pulse.
E19	N15 Output N · control	N01 Mission controller	.self recursion	`.self · validated input N → N+1`	Validated output becomes input N+1 through ⊕.

Mermaid export

```
flowchart TD
  subgraph L_spine["Admission spine"]
    N01["N01 · Mission controller"]:::auth
    N02["N02 · Asymmetric allocation"]:::auth
    N04["N04 · Parallel worker lanes"]:::proc
    N05["N05 · Preflight + pre-match cache"]:::proc
    N06{"N06 · First completed material red?"}:::dec
    N09["N09 · Exact-head final admission"]:::proc
    N10{"N10 · All non-waivable gates green?"}:::dec
    N13["N13 · Single-flight merge controller"]:::pass
    N14["N14 · Exact-main CI · metrics"]:::pass
    N15(["N15 · Output N · control"]):::auth
  end
  subgraph L_ev["Evidence · hold"]
    N03["N03 · Async I/O"]:::proc
    N11["N11 · HOLD"]:::hold
    N12[("N12 · REX · KEB evidence")]:::evid
  end
  subgraph L_rep["Repair loop"]
    N07["N07 · Isolated repair"]:::rep
    N08["N08 · Impacted retest"]:::rep
  end
  N01 --> N02
  N02 --> N04
  N02 -->|"async · cheap"| N03
  N04 --> N05
  N03 -.->|"pre-match"| N05
  N05 --> N06
  N06 -->|"yes"| N07
  N07 --> N08
  N08 -->|"verify"| N09
  N06 -->|"no red"| N09
  N09 --> N10
  N10 -.->|"no"| N11
  N10 -->|"yes"| N13
  N13 --> N14
  N14 --> N15
  N11 -.->|"hold record"| N12
  N12 -.->|"reuse proven evidence"| N03
  N11 -.->|"material change → next pulse"| N01
  N15 ==>|".self · validated input N → N+1"| N01
  classDef auth fill:#56287a,color:#fff,stroke:#3f1c5c
  classDef proc fill:#eee7f7,color:#452366,stroke:#9879b8
  classDef dec fill:#fff,color:#1c1626,stroke:#5a2a86,stroke-dasharray:5 3
  classDef rep fill:#fce6eb,color:#8f2747,stroke:#cf5f7d
  classDef pass fill:#dff4e9,color:#145c43,stroke:#3f9b6c
  classDef hold fill:#fff1d9,color:#7c5118,stroke:#c98f2e
  classDef evid fill:#def1f2,color:#135458,stroke:#2f8f95
  linkStyle 9,12,13,14 stroke:#3f9b6c,stroke-width:2px
  linkStyle 6,7 stroke:#cf5f7d,stroke-width:2px
  linkStyle 11,17 stroke:#c98f2e,stroke-width:2px,stroke-dasharray:6 4
  linkStyle 4,15,16 stroke:#2f8f95,stroke-dasharray:2 4
  linkStyle 18 stroke:#5a2a86,stroke-width:3px
```

4 · Recursive SIPOC

A standard SIPOC runs once, left to right. With `.self`, the customer of iteration N becomes the supplier of iteration N+1, and the next input is a governed reconciliation, not a concatenation.



State transition

$$I_{N+1} = O_N \oplus \Delta E_N \oplus C_{N+1}$$

O_N	Last output: receipts, check states, ranked findings.
ΔE_N	Newly observed evidence: completed checks, runner behaviour, new findings.
C_{N+1}	Newly applicable constraints: gates, policies, review rules.
$\oplus$	Governed reconciliation. Superseded evidence is retired, conflicts go to HOLD, nothing is appended blindly.

Confidence is not admission

A passing local preflight raises confidence for the next iteration. Only exact-head GitHub checks and reviews move a PR through the admission gate.

5 · Lanes and dock

Ten execution lanes grouped by what they contribute. Each lane keeps its existing canonical owner; Code Purple only routes between them. "Show on pulse" traces the lane onto the execution graph.

PROTOCOL
Code Purple

TIER 1 — Discover

Global lookup

`GLOB.yaml · global mesh census`

Discover authorities and available capabilities.

`CODEX`

Dock interface

`.mesh/MESH_DOCK.yaml`

Resolve producer, consumer, receipts and scope.

`mesh`

TIER 2 — Verify

Runtime and self-test

`scripts/testpilot_preflight.py`

Fast preflight, smoke and testflight.

`ABACUS`

Lint and configuration

`pre-commit · Ruff · YAML/JSON/TOML`

Detect deterministic defects early.

`both`

Coverage

`ci-cd-tests.yml · MIP B0 coverage evidence`

Line + branch per test; breadth, depth and per-area census; floor ratchet (T16).

`ABACUS`

Security

`Bandit · Semgrep · SARIF`

Typed findings. No fabricated security PASS.

`both`

TIER 3 — Analyse and remember

Derived analysis

`DOW`

Rank findings and return scoped dispositions.

`ABACUS`

Semantic and provenance

`KEB · REX`

Evidence identity, recurrence and reusable lessons.

`CODEX`

TIER 4 — Authorise and deliver

QPS domain

`cryoplant-project controlled sources`

Engineering authority over acceptance stays here.

`cryoplant-project`

Delivery

`GitHub Actions · PR Feedback Gate · Pages`

Exact-head admission and post-merge proof.

`GitHub`

Two meanings of "dock"

`logical` The mesh contract in `MESH_DOCK.yaml`: who produces, who consumes, which receipts count, what scope applies.

`runtime` Container networking: ports a devcontainer or compose service exposes. Configuration alone proves nothing about a running service.

Runtime dock status ladder

SURFACE	CONFIGURED	LISTENING	PUBLISHED	PROBED
ABACUS devcontainer `8000 · 8080 · 5000 forwarded`	●	◌	◌	◌
CODEX compose validator `no published ports`	●	◌	—	◌

● verified ◌ unproven — not declared

6 · Admission board

The inbox shows an execution bottleneck rather than a coding backlog. Each red gets one of four causes before anyone repairs it, so a runner outage is never fixed as a source defect.

RED IN SCOPE
4
ABACUS #1856 #1857 #1858 #1859

READY TO MERGE
0
ABACUS main idle since #1838 (10-06)

OPEN ABACUS PRS
20
all held by the Codex review gate

MERGED TODAY
1
CODEX #984, with 4 red checks

Failure classes

Source defect
Introduced by this PR. Repair in the PR.

Inherited quality debt
Already on main. Ratchet new debt; keep full-main enforcement.

Review capacity
Codex review or security review unavailable. Owner fallback, never self-approval.

Runner or infrastructure
Not a code fault. Retry or fix the runner, then re-read.

Pull requests in scope · read back 17:55 CEST

ABACUS #1858 — P0 · gate FAILURE

Owner-attested review fallback

Head e7616e62 (17:52): regenerated rationalization doc, policy-and-inventory green. Remaining reds (Code Quality, Security Scanning; Coverage and Full Red Proof still running at 17:55) all fail nightly on ABACUS main: inherited debt. Threads resolved; the follow-up fixes are already in the PR (a7f623b5, 14430046). Gate pending on Codex reviews, so landing needs the owner break-glass.

ABACUS #1859 — 4 red

PR-level new-debt ratchet

Head 6fac58cc. Red: Code Quality Checks, Full Red Proof Summary, Security Scanning, validate-scripts. Class not yet assigned. Distinct control from #1858; must not weaken full-main enforcement.

ABACUS #1857 — 4 red

Fix syntax error in diagnostic_kpi_probe.py

Head e7b45912. Red: Code Coverage, Code Quality Checks, Full Red Proof Summary, Security Scanning. Likely inherited debt (the same reds as #1856); class not yet confirmed.

ABACUS #1856 — 3 red

Lossless branch hygiene burn-down

Head f11d8044. Red: Code Coverage, Code Quality Checks, Full Red Proof Summary. Same reds recur across PRs, which points to inherited debt; #1859 targets exactly this.

CODEX #984 — merged 12:16

Global Code Purple seed

Merged with 4 red checks on head 90e3396a (ruff, governance-gate ×2, validate). CODEX main has no required-check ruleset, so nothing stopped it. Post-merge 768c9e35: 38 pass, 2 fail (validate, scheduled osv-scan). Current main 6edf28b7 green.

Burndown

#	ITEM	STATUS	NEXT ACTION	OWNER	TARGET (PROPOSED)
1	Land ABACUS #1858	in progress	Done: threads resolved (17:20), #1878/#1879 opened, rationalization doc pushed (e7616e62, policy-and-inventory green). Next: owner accepts the 4 inherited reds and adds the one-time bypass; Claude squash-merges and reads back; owner restores the ruleset.	GBO (ruleset, merge) · Claude (threads, issues, readback)	2026-10-13
2	Follow-up hardening for #1858	opened	#1878 gate pending on `pull_request: synchronize`; #1879 correlate outage reply with a current-head request.	Open for implementation	2026-10-13
3	CODEX post-merge reds	classified	validate fixed on main at f9ae4676. OSV-Scanner fails on every main push since before #984: inherited, needs its own fix.	Claude	2026-10-13
4	CODEX admission gap	decision	Decide whether CODEX main gets a required-check ruleset like ABACUS.	GBO	2026-10-14
5	Classify reds on #1856, #1857, #1859	confirmed	Code Coverage, Code Quality, Full Red Proof and Security Scanning fail in every nightly run on ABACUS main since at least 7 Oct: inherited debt.	Claude	2026-10-14
6	Runnable `.self` harness	todo	Emit census JSON (contract in the Census tab), SARIF findings and exact-commit receipts.	Claude	2026-10-16
7	Scheduled readback pulse (T12)	todo	Read back tracked PRs and both mains on a schedule; update this page's data.	Claude	2026-10-16
8	Docker smoke harness, live port probing	not built	After the harness contract is stable.	Claude	2026-10-23
9	Cross-repository adoption (T09)	gated	Adopt the seed in ABACUS once #1858 has landed.	—	gate: item 1
–	QPS completed lanes	guard	Preserve; no regression from Code Purple changes.	—	ongoing

7 · Lift and method

Not lifted. Merge flow returned when #1858 merged on 12 October, but the ruleset was emptied during the break-glass and never restored: since then no merge has passed a green PR Feedback Gate (EVERRED root EVR-ABACUS-0001). These criteria are read from GitHub by `kit/readback.py`; the eight steps below are the reusable method, adapted per incident through one file, `kit/incident.yaml`.

OPEN ABACUS PRS
20 → 2
declaration → 13 Oct 17:43

MERGES IN 23 H
44
after 6 days with none

COVERAGE
40.3%
from 36.9%; 1,247 tests

RULESET RULES
0
emptied 12 Oct 17:58; main unprotected

ID	LIFT CRITERION	MET	EVIDENCE
L1	Unblocker PR merged	yes	#1858 merged 2026-10-12 18:17 CEST (e10e39d3)
L2	Break-glass bypass removed	yes	Ruleset 24494381 active, 0 bypass actors
L3	Merge flow restored (≥ 5 merges since the unblocker)	yes	43 merges after #1858
L4	Backlog ≤ 25% of baseline	yes	2 open (baseline 20)

Worked

- Readback before action showed #1858 could not admit itself and #984 had merged red.
- Classifying against main's nightly run proved four reds inherited.
- The smallest fix (a 3-line regenerated doc) cleared the only red the PR caused.
- Owner/agent split: owner bypass, merge and restore; agent threads, issues, fix, readback.

Change next time

- Run the readback every pulse and on a schedule; snapshots went stale twice in an hour.
- Re-read the PR head before writing a decision on it.
- Decide on a required-check ruleset for CODEX (#984 merged with 4 reds).
- Break glass with a bypass actor, never by deleting rules; the lift check now compares actual rules to expected rules (it passed falsely on 13 Oct).
- Burn down the four inherited reds; coverage W0 due 16 Oct.

The method in eight steps

STEP	NAME	ENTRY	ACTION	WHO
0	**Declare**	Merge flow stalled, the same checks red across PRs, or a required reviewer unavailable	Write incident.yaml	Owner
1	**Read back**	incident.yaml exists	Run readback.py; no action before the first census	Agent
2	**Classify every red**	Census on exact heads	Inherited · source defect · review capacity · runner or infra	Agent
3	**Find the unblocker**	Classes known	Smallest change that lets the gate admit again; prove it cannot admit itself	Agent + owner
4	**Repair source defects only**	Unblocker chosen	Fix, impacted retest, exact-head readback	Agent
5	**Admit or break glass**	Only inherited or capacity reds left	Owner writes residual risk, one-time bypass, merge, remove bypass	Owner
6	**Lift check**	Unblocker merged	readback.py evaluates the L-criteria	Agent
7	**Close out**	Lifted	Verify follow-ups on main and close; REX lesson; inherited reds become dated burn-down items	Agent + owner

Toolkit files: incident.yaml · readback.py · after_action.md · code-purple.md